

Codeforces and CodeBERT Chatbot for Competitive Programming

Assignment 2

January 5, 2025

1 Overview

Time to put pre-scraped Codeforces problems, their editorials, and metadata to use by creating an intelligent assistant that can help users understand and solve programming problems.

2 Assignment Structure

```
1 project/
2
3     data/
4         problems/           # Scraped problem data
5         editorials/        # Problem editorials
6         metadata/          # Problem metadata
7
8     src/
9         embeddings.py       # CodeBERT embedding generation
10        vectorstore.py      # Vector database management
11        retriever.py        # RAG implementation
12        chatbot.py          # Chatbot interface
13
14        notebooks/
15            examples.ipynb   # Usage examples
```

3 Implementation Components

3.1 Embedding Generation

3.1.1 Understanding CodeBERT Embeddings

Before implementation, it's important to understand how CodeBERT processes text:

1. **Tokenization:** Text is split into tokens
2. **Encoding:** Tokens are converted to IDs
3. **Embedding:** IDs are transformed into vector representations

3.1.2 Implementation Steps

To do!

3.1.3 Testing the Embedder

Here's how to test your implementation:

```
1 # Test single embedding
2 embedder = CodeBERTEmbedder()
3 test_text = "Find the maximum subarray sum"
4 embedding = embedder.generate_embedding(test_text)
5 print(f"Embedding shape: {embedding.shape}")
6
7 # Test batch embedding
8 texts = [
9     "Find the maximum subarray sum",
10    "Implement a segment tree",
11    "Solve the knapsack problem"
12 ]
13 embeddings = embedder.batch_generate_embeddings(texts)
14 print(f"Batch embeddings shape: {embeddings.shape}")
```

3.2 Vector Store Implementation

3.2.1 Understanding FAISS Index

FAISS provides efficient similarity search:

- IndexFlatIP: Exact inner product search
- IndexFlatL2: Exact L2 distance search
- IndexIVFFlat: Approximate search with better speed

3.3 RAG Implementation

3.3.1 Key Components of RAG

Understanding the RAG process:

1. Query Understanding and Context Retrieval
2. Response Generation

3.3.2 Implementation

Use the hints to make a systematic RAGRetriever.

3.4 Chatbot Integration

3.4.1 Understanding the Chat Flow

The chatbot process involves:

1. Receiving user query and retrieving relevant context
2. Generating response and managing conversation history

3.4.2 Implementation

Final step.

4 Example Usage

```
1 # Initialize components
2 embedder = CodeBERTEmbedder()
3 vector_store = VectorStore()
4 retriever = RAGRetriever(embedder, vector_store)
5
6 # Create chatbot
7 system_message = """I am solving a Competitive Programming problem, and I need
8     help understanding its editorial.
9 Answer my questions regarding the editorial.
10 Let me know if I'm misunderstanding anything.
11 Do not write or debug code."""
12 chatbot = CPChatbot(retriever, system_message)
13
14 # Chat
15 response = chatbot.chat("How do I solve problem C from Contest#792?")
```

5 Submission Requirements

1. GitHub repository with complete code
2. README with setup and usage instructions
3. Example notebook demonstrating functionality